ReconSage

An nmap-Correlated Recon-to-Recommendation Platform for Authorized Penetration Testing

Project report | ReconSage v0.1.0 | November 2026

Abstract

External and internal penetration tests begin with network reconnaissance, but the gap between raw scanner output and an actionable attack plan is bridged manually, repeatedly, by the tester. ReconSage is an open, locally run tool that closes part of that gap: it parses nmap scan results, matches each discovered service against a knowledge base of manual checks, version-specific CVE hints and MITRE ATT&CK technique identifiers, and correlates combinations of services into host-level findings such as Active Directory Domain Controller detection. Findings are rendered as a prioritized Markdown report. The tool deliberately stops at recommendation: it does not execute exploits, brute-force credentials, or otherwise act on a target beyond the scan the operator explicitly requests. This report describes ReconSage's motivation, design, detection methodology, and a validation run against a real, deliberately vulnerable host (Metasploitable2) in an isolated lab network, and states plainly what that run does and does not demonstrate.

1. Introduction and motivation

A single nmap service-version scan against a modestly sized host can return a dozen or more open ports, each requiring the tester to recall or look up what that service typically means for attack surface: which manual checks apply, whether the detected version matches a known CVE, and whether the combination of services present signals something larger than any individual port (for example, Kerberos, LDAP and SMB together indicating a Domain Controller). Experienced operators perform this correlation automatically; less experienced testers perform it by repeated manual lookup, which is slow and inconsistent across engagements.

ReconSage automates the lookup step without automating the judgment step. It was built around a specific design constraint: every output line is a lead for a human to investigate, never an action taken against the target. This constraint was treated as a requirement throughout development, not an afterthought, because it determines both the tool's safety profile and its value as a learning instrument -- a junior tester who is handed conclusions learns less than one who is handed a prioritized list of things to go verify.

2. Objectives and scope

Objectives

- Reduce the time between raw scan output and a prioritized, actionable checklist.
- Encode manual-check knowledge in a data file (YAML), not program logic, so the knowledge base can be extended without touching code.

- Detect host-level patterns that only emerge from combinations of services, not any single port in isolation.
- Preserve a clear line between recommendation and action: no exploitation, no credential attacks, no traffic beyond the scan the operator runs.

In scope

- Parsing nmap XML (-oX) output into structured host/service data.
- Per-service matching against a manual-checks and CVE-hint knowledge base.
- Cross-service pattern correlation (e.g. Domain Controller fingerprinting, legacy-protocol clustering, directly exposed databases).
- Markdown report generation, severity-ranked.

Out of scope

- Exploitation, credential brute-forcing, or any automated action against a target.
- Web application content discovery and vulnerability scanning (left to dedicated tools such as gobuster, ffuf, Burp Suite).
- Testing of any system the operator does not own or lack explicit written authorization to test.

3. Related work

Automated reconnaissance aggregators such as AutoRecon and Legion run a battery of service-specific enumeration tools against discovered ports and organize the resulting output for the tester to review. ReconSage differs in emphasis: rather than running additional enumeration tools itself, it focuses on correlating nmap's own service-detection output against a structured, auditable knowledge base, and on detecting host-level patterns across services rather than treating each port independently. This keeps the tool's dependency surface small (it shells out to nmap only, and optionally reads an existing scan file) and keeps the entire "intelligence" layer in two human-readable YAML files that a tester can read, audit, and extend without reading Python.

4. System design

4.1 Pipeline

Each run flows through the same linear stages:

```
nmap scan (-oX)  -->  parser  -->  rules engine  -->  correlation engine  -->
report
```

4.2 Components

Component	Role
parser.py	Parses nmap XML into Host/Service dataclasses (IP, hostname, per-port protocol, service name, product, version).
rules_engine.py	Matches each Service against services.yaml; emits a Finding per matched check and per CVE-hint banner match.
correlate.py	Matches the full set of service names present on a host against patterns.yaml; emits

Component	Role
	host-level Findings for multi-service combinations.
report.py	Renders all Findings, sorted by severity, into a Markdown report with an open-services table per host.
cli.py	scan (drives nmap directly) and analyze (reads an existing -oX file) subcommands.

4.3 Data model

A Host holds zero or more Service records and two Finding lists: per-service findings produced by the rules engine, and pattern findings produced by the correlation engine. A Finding carries a title, a free-text detail, an optional CVE identifier, an optional MITRE ATT&CK technique identifier, and a severity level (info, low, medium, high).

5. Detection methodology

5.1 Per-service rules

Each entry in services.yaml matches a Service by its nmap-reported service name, by port number as a fallback, or both. A matching entry contributes a Finding listing manual checks to try. If the service's product/version banner contains a known-vulnerable substring (case-insensitive), a second Finding is emitted naming the matching CVE. This two-tier design -- general checks plus specific version matches -- keeps the common case (service identified, no specific CVE known) useful while surfacing high-confidence CVE matches distinctly.

5.2 Cross-service pattern correlation

patterns.yaml rules examine the full set of service names present on a host, not individual ports. Four match strategies are supported: all_of (every named service must be present), any_of (at least one), any_of_groups (every service within at least one named group must be present), and any_n_of (at least n of a named set). The Domain Controller pattern, for example, fires when Kerberos, LDAP and SMB are all present together -- a signal that is invisible when each port is evaluated independently.

5.3 Severity and ordering

Findings are ordered high, medium, low, info for report readability, so the highest-value leads appear first regardless of which port or pattern produced them.

6. Rule library at time of this report

Artifact	Entries	Notes
services.yaml	24 service rules	Covers common TCP services: ftp, ssh, telnet, smtp, dns, http/https, pop3, imap, rpcbind, snmp, ldap, smb, mysql, mssql, rdp, vnc, winrm, redis, kerberos, elasticsearch.
patterns.yaml	5 correlation rules	Domain Controller (two match variants), Windows file + remote-management co-exposure, legacy cleartext protocol clustering, directly exposed database services.

The rule library is intentionally not exhaustive at this stage; Section 9 documents specific gaps found during live testing and the process by which they are closed.

7. Evaluation methodology

Validation in this report is split into two parts: a structural verification using an automated test suite against a hand-built sample scan (Section 8), and an empirical run against a real, intentionally vulnerable host to test the rule library's real-world coverage (Section 9).

8. Implementation and engineering quality

- Python package, standard-library XML parsing; PyYAML is the only runtime dependency.
- CLI with scan (drives nmap -sV -sC directly) and analyze (reads an existing -oX file) subcommands.
- Automated test suite: 5 tests covering XML parsing correctness, CVE-hint matching, duplicate-finding regression (SMB on port 445 under two overlapping rule matches), Domain Controller pattern firing, and correct non-firing on a host that is not a Domain Controller. All 5 pass at time of writing.
- Packaged with pyproject.toml for pip/pipx installation; MIT licensed.

9. Validation

This section reports what was verified. It validates the tooling's behavior against real and synthetic scan data; it is not a claim about the security of any specific target class.

9.1 A duplicate-finding defect found and fixed

Initial testing against the bundled sample scan surfaced a defect: a host with SMB exposed on port 445 produced two near-identical findings for the same port, because two separate rule entries in services.yaml (one matching by service name, one by port number) both matched the same service. The fix merges service-name matching into a single list-valued field per rule and consolidates the two SMB entries into one. A regression test (test_smb_finding_not_duplicated) was added to the suite to prevent recurrence.

9.2 Live validation against Metasploitable2

Protocol: Metasploitable2 was run in an isolated VirtualBox host-only network alongside a Kali Linux attacker VM. Connectivity was confirmed with ICMP (ping -c 3) before scanning. ReconSage's scan subcommand was run directly against the target, which internally executes nmap -sV -sC -oX and analyzes the result.

```
reconsage scan 192.168.56.102 -o report_live.md
```

Result: 23 open TCP ports were identified. The rules engine and correlation engine together produced 16 findings across 11 distinct ports and 3 host-level patterns (cleartext-protocol clustering, directly exposed database, and the two independent FTP-anonymous-login leads). The full, unedited report is reproduced in Appendix C.

9.3 Coverage gaps identified by this run

Comparing the full set of 23 open ports against the 16 generated findings identified services with no corresponding rule at the time of the run:

Port	Service	Significance
1524	bindshell	nmap's own banner read "Metasploitable root shell" -- an unauthenticated root shell. The single highest-value finding on the host produced no corresponding ReconSage output.
6667	irc (UnrealIRCd)	This exact build corresponds to a publicly documented backdoor (CVE-2010-2075, unauthenticated remote command execution).
512 / 513 / 514	exec / login / shell (r-services)	Legacy trust-based remote access services, historically exploitable with no credentials.
2049	nfs	Worth checking for world-readable or world-writable exports; no corresponding rule existed.
1099	java-rmi	Older RMI registries carry known deserialization RCE issues.
5432	postgresql	Present in the open-services table but not yet included in the directly-exposed-database pattern rule.

This outcome is treated as the expected and useful result of testing a recommendation engine against a real target, not as a failure: it demonstrates that the rule library was incomplete in a way the bundled synthetic sample data could not reveal, and that the fix is a YAML addition rather than a code change.

10. Results

Table 2 reproduces the severity distribution of findings from the Section 9.2 run, prior to closing the gaps identified in Section 9.3.

Severity	Findings	Representative example
High [!!!]	6	CVE-2011-2523 (vsftpd 2.3.4 backdoor); SMB null-session checks on ports 139 and 445; legacy cleartext protocol clustering; directly exposed database (MySQL).
Medium [!!]	6	Anonymous FTP login checks on ports 21 and 2121; web content discovery on ports 80 and 8180; MySQL/VNC weak-credential checks.
Low [!]	4	SSH authentication method review; SMTP user enumeration; DNS zone-transfer check; RPC service enumeration.

11. Interpreting results

- A finding appearing in the report is a lead for manual verification, not a confirmed vulnerability; version banners can be inaccurate or intentionally spoofed.
- Absence of a finding for an open port means the rule library has no entry for that service, not that the service is safe -- Section 9.3 demonstrates this directly.
- A fired pattern finding (e.g. Domain Controller) should raise, not replace, the priority of manual AD enumeration on that host.
- Severity labels reflect the rule author's judgment about the value of investigating a lead, not a measured exploitability score.

12. Limitations and threats to validity

- The rule library's coverage is finite and was, at the time of this report, demonstrated incomplete even against a well-known, widely used training target.

- CVE-hint matching is a case-insensitive substring match against the banner string; it will miss patched services that retain an old banner string, and will not catch vulnerabilities with no distinctive banner signature.
- The tool performs no verification of its own findings; every lead requires manual confirmation before being acted on.
- Pattern correlation operates on nmap's service-name field only; a service running on a non-standard port with a generic or unrecognized banner may not be matched by name-based or port-based rules.
- Validation to date covers one real target (Metasploitable2) and one synthetic sample; broader validation across a wider range of host types has not yet been performed.

13. Ethics and responsible use

ReconSage is built for use only against systems the operator owns or is explicitly authorized to test: a personal lab, a CTF or training platform, or an engagement under signed authorization. It performs no exploitation, no credential attacks, and generates no traffic toward a target beyond the scan the operator explicitly initiates. The tool's scope was deliberately bounded at recommendation rather than action during design, for two stated reasons: a published auto-exploitation tool invites a different and higher level of scrutiny than a recon-correlation tool, and stopping at recommendation preserves the verification and judgment step as something the operator practices rather than something the tool performs for them.

14. Future work

- Extend services.yaml to cover the gaps identified in Section 9.3 (bindshell, IRC/UnrealIRCd, r-services, NFS, Java RMI) and add postgresql to the database-exposure pattern.
- Optional live CVE lookup against the NVD API for services with no static YAML entry, as a fallback to the bundled rule set.
- HTML report output alongside Markdown, for engagements that require a client-facing deliverable.
- SQLite-backed scan history to allow diffing two scans of the same host over time.
- Broaden validation to additional deliberately vulnerable targets (e.g. additional VulnHub images, a small Active Directory lab) to further stress-test pattern correlation rules.

15. Conclusion

ReconSage demonstrates that a substantial share of the manual lookup work in network reconnaissance -- mapping an open port and version banner to the checks, CVEs and ATT&CK techniques an experienced operator would reach for -- can be encoded in an auditable, extensible data file and automated without automating the judgment calls that follow. Its validation against a real target surfaced genuine coverage gaps rather than merely confirming expected behavior, which is treated here as evidence the validation method was meaningful rather than as a shortcoming of the tool. The project's explicit boundary -- recommendation, not action -- is maintained throughout its design and is the basis on which its further extension is planned.

Appendix A. Reproducing the results

```
git clone <repo-url>
cd reconsage
pip install -e .
reconsage analyze sample_data/sample_scan.xml

# live validation (requires an authorized target, e.g. Metasploitable2)
reconsage scan <target-ip> -o report_live.md

# test suite
pip install pytest
pytest tests/
```

Appendix B. Glossary

Term	Meaning
Finding	One actionable recommendation attached to a service or to a host-level pattern.
Pattern correlation	Matching combinations of open services on a host against rules that span more than one port.
Severity (ReconSage)	The rule author's judgment of how worthwhile a lead is to investigate; not a measured exploitability score.
Recon-to-recommendation	This project's design boundary: the tool produces leads for a human to verify, and performs no exploitation, credential attack, or other action on a target.

Appendix C. Full report from the Section 9.2 validation run

Target: Metasploitable2, isolated VirtualBox host-only network. Reproduced verbatim from report_live.md.

```
# ReconSage Report

> Recon-to-recommendation only. Every item below is a lead to investigate
> manually -- nothing here was auto-exploited.

## 192.168.56.102

### Open services: 23 ports, including ftp (21, 2121), ssh (22), telnet (23),
### smtp (25), dns (53), http (80, 8180), rpcbind (111), smb (139, 445),
### exec/login/shell (512-514), java-rmi (1099), bindshell (1524), nfs (2049),
### mysql (3306), postgresql (5432), vnc (5900), X11 (6000), irc (6667),
### ajp13 (8009).

### Prioritized findings (16 total; severity-sorted)
[!!!] CVE-2011-2523 -- 21/tcp vsftpd 2.3.4 backdoored build
[!!!] T1040 -- 23/tcp telnet, cleartext by design
[!!!] T1210 -- 139/tcp and 445/tcp smb, null-session and signing checks
[!!!] T1040 -- multiple cleartext/legacy protocols exposed together
[!!!] T1190 -- database service (mysql) reachable without an app in front
[!!] T1190 -- 21/tcp and 2121/tcp ftp, anonymous login checks
[!!] T1190 -- 80/tcp and 8180/tcp http, content discovery and fingerprinting
[!!] T1110 -- 3306/tcp mysql, default/weak credential checks
[!!] T1110 -- 5900/tcp vnc, no-password check
[!] T1110 -- 22/tcp ssh, auth method review
[!] T1589 -- 25/tcp smtp, user enumeration
```

```
[!] T1590 -- 53/tcp dns, zone transfer check
[!] T1590 -- 111/tcp rpcbind, service enumeration

Full table-formatted version: RECONSAGE_REPORT.md in the project repository.
```

References

OWASP Foundation. OWASP Top 10 for LLM Applications and related application-security guidance on reconnaissance and attack-surface mapping.

MITRE ATT&CK. Enterprise Matrix -- technique identifiers referenced throughout this report (T1040, T1087, T1110, T1190, T1210, T1482, T1558, T1589, T1590).

Rapid7 / community. Metasploitable2 -- intentionally vulnerable Linux target used for the Section 9.2 validation run.

Tib3rius and contributors. AutoRecon -- multi-threaded network reconnaissance tool; cited in Section 3 as related prior work.

GoVanguard. Legion -- semi-automated network penetration testing framework; cited in Section 3 as related prior work.